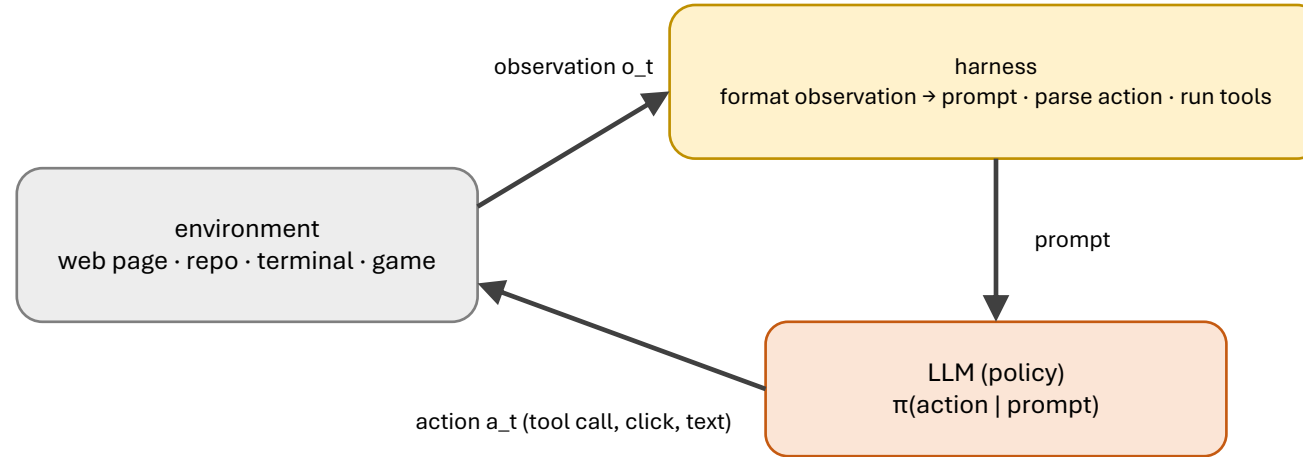


From a model to an agent

- Lectures 11–12 trained the **model**; products (Claude Code, Codex, computer-use agents) are the model plus a **harness**: the loop, tools, memory, permissions and checks around it
- Claim of the year (YC Paper Club, 2026: *Why the harness matters more than the model*): with the same model, the harness decides what gets solved
- Plan: what an LLM agent is → what a harness contains → verification and search at inference time (Princeton COS 597R) → **ARC-AGI-3**, the 2026 benchmark where frontier models score below 1% and harness research is the game

What is an LLM agent?



same loop as Lecture 11a; the reward (if any) is a task outcome: tests pass, level solved, user approves

An agent is a policy in an environment (Lecture 11a): observation in, action out, repeat

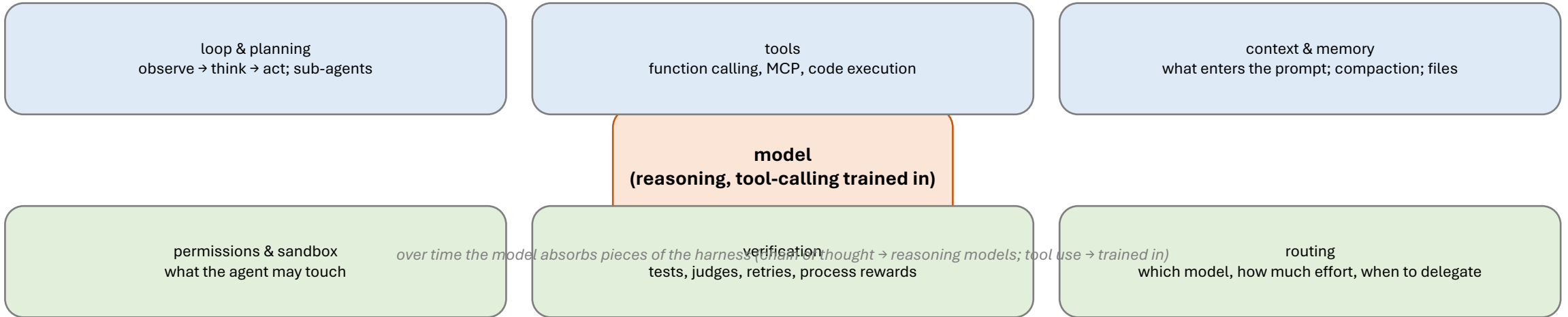
In an LLM agent the model chooses the action; the **harness** turns observations (DOM, files, screenshots, game frames) into a prompt and parses the model's text into an action

Examples: web agents (click/type), SWE-agent (open, edit, run tests), computer-use agents (screenshots → clicks and keys), game agents (ARC-AGI-3)

The agent–computer interface matters as much as the model

- **SWE-agent** (2024): the environment exposes a small action API (open file, goto line, search, edit) and formats outputs for the model — replacing a raw shell raised success rates for the same model
- **Function calling**: the API describes tools as JSON schemas; the model emits structured calls; the harness executes them and returns results — standardized across vendors by the Model Context Protocol (Nov 2024)
- **Think before acting**: a chain-of-thought field before each action (Lecture 12's reasoning, at every step)
- **Computer use**: the general interface — screenshots in, mouse/keyboard out — slower but works for any software

The harness: everything around the model



Loop and planning, tools, context management (what enters the prompt, compaction, sub-agents), permissions and sandboxing, verification (tests, judges, retries), routing (model choice, effort, delegation)

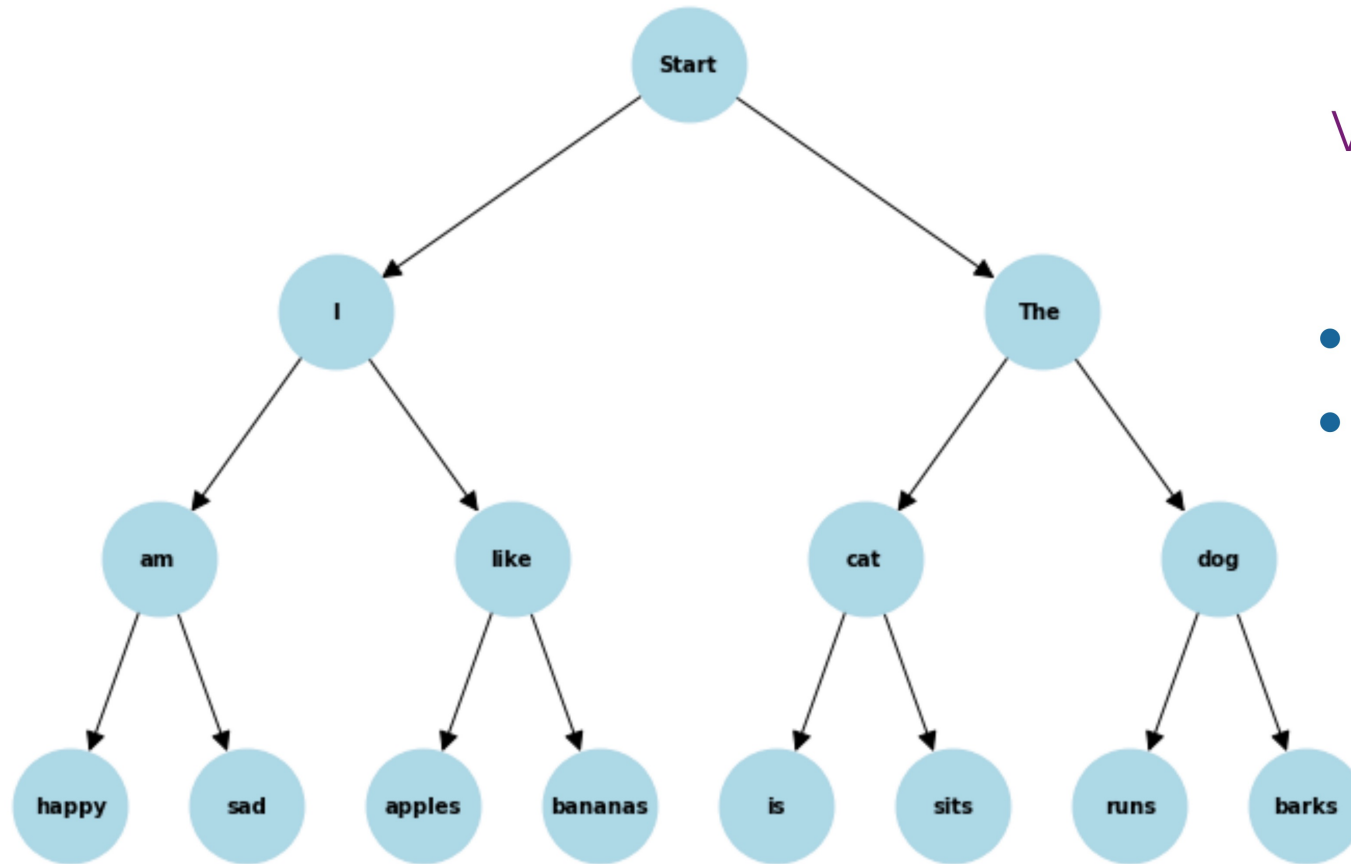
Harnesses migrate into the model: chain-of-thought began as a third-party harness around GPT-3, became a first-party harness at OpenAI, and shipped as the o1 reasoning models (ARC Prize report, 2026)

Raschka (2026): as models improve, Claude Code and Manus **removed** scaffolding — but orchestrating agents that spawn sub-agents makes the overall system harder to supervise

Same model, different harness → different leaderboard numbers (SWE-bench entries list model **and** scaffold; ARC-AGI-3 keeps harnessed agents on a separate community leaderboard)

LLM Generation = “Policy”

LLM as Next-Word Policy Generator



Value function = “quality” of response

- How do we learn a good value function?
- Can we use policy rollouts at inference time to improve value of the generated response?

Pass@k viewed as inference-time search

Pass@k

“Generate k answers to the query; pick the best.”

(e.g., Pass@5 often much better than Pass@1)

Evaluating “Best” requires a reward Model for correctness/quality of math Q&A

PassRatio@K (when answer is a single word or number):
Generate K answers and determine the fraction that are correct

Hurdle in learning reward function using naive SFT on human-labeled examples:

- Even strong models can’t reliably tell good answers from wrong
- Using wrong labels in SFT hurts performance super proportionately

Today’s examples (GSM8K and MATH) have autoverifiable answers

Process Supervision (PRM)

Given: GSM8K question, CoT answer, Final Answer

Goal: Give a reward 0/ 1/ -1 to each step i.e., (neutral/positive/negative)

The denominator of a fraction is 7 less than 3 times the numerator.

If the fraction is equivalent to $\frac{2}{5}$, what is the numerator of the fraction?

   Let's call the numerator x .

   So the denominator is $3x-7$.

   We know that $\frac{x}{3x-7} = \frac{2}{5}$.

   So $5x = 2(3x-7)$.

   $5x = 6x - 14$.

   So $x = 7$.

PRM800K: Dataset contains 800K step-level labels provided by human raters across 75K solutions to 12K problems (MATH 8K training set + 4K test questions).

Search + verification is what a harness adds at test time

- Sample many candidates, **verify** (unit tests, math checkers, a reward model), keep the best — pass@k turns into real accuracy only with a good verifier
- Reasoning models (Lecture 12) internalized the 'think first' harness; agents still externalize the rest: tools, retries, tests, sub-agents
- Lecture 12's verifiable rewards are the training-time version of the same idea; here they run at inference time
- Next: a benchmark built so that neither the model alone nor memorized harnesses can solve it

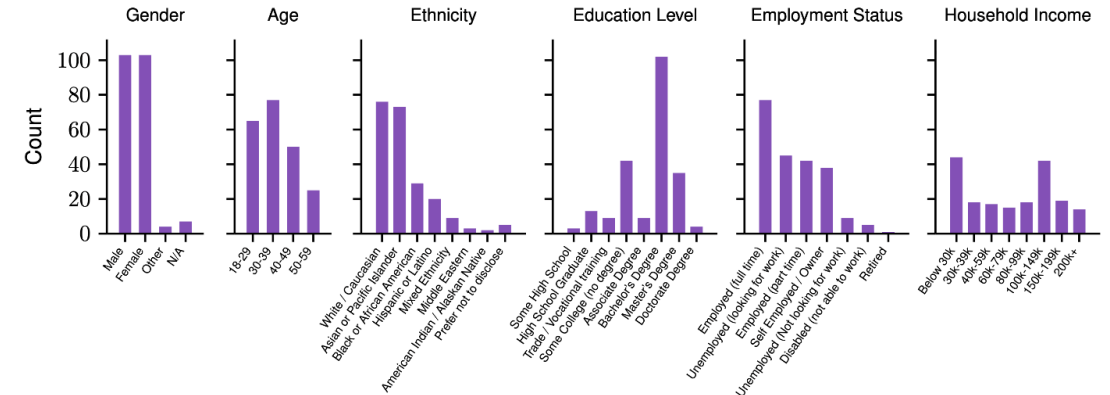
ARC-AGI-3 in one slide

- Hand-designed, turn-based **game environments** (each a series of levels) — no instructions, no rules, no stated goal
- The agent must **explore**, infer the goal, build a model of the dynamics and plan action sequences; only Core Knowledge priors, no language or world knowledge
- Difficulty calibrated with human testers: **humans solve 100%**; frontier AI systems score **below 1%** (March 2026)
- ARC Prize 2026: over \$2M in prizes; a Kaggle competition on private environments (3 public games ft09, ls20, vc33 for development)



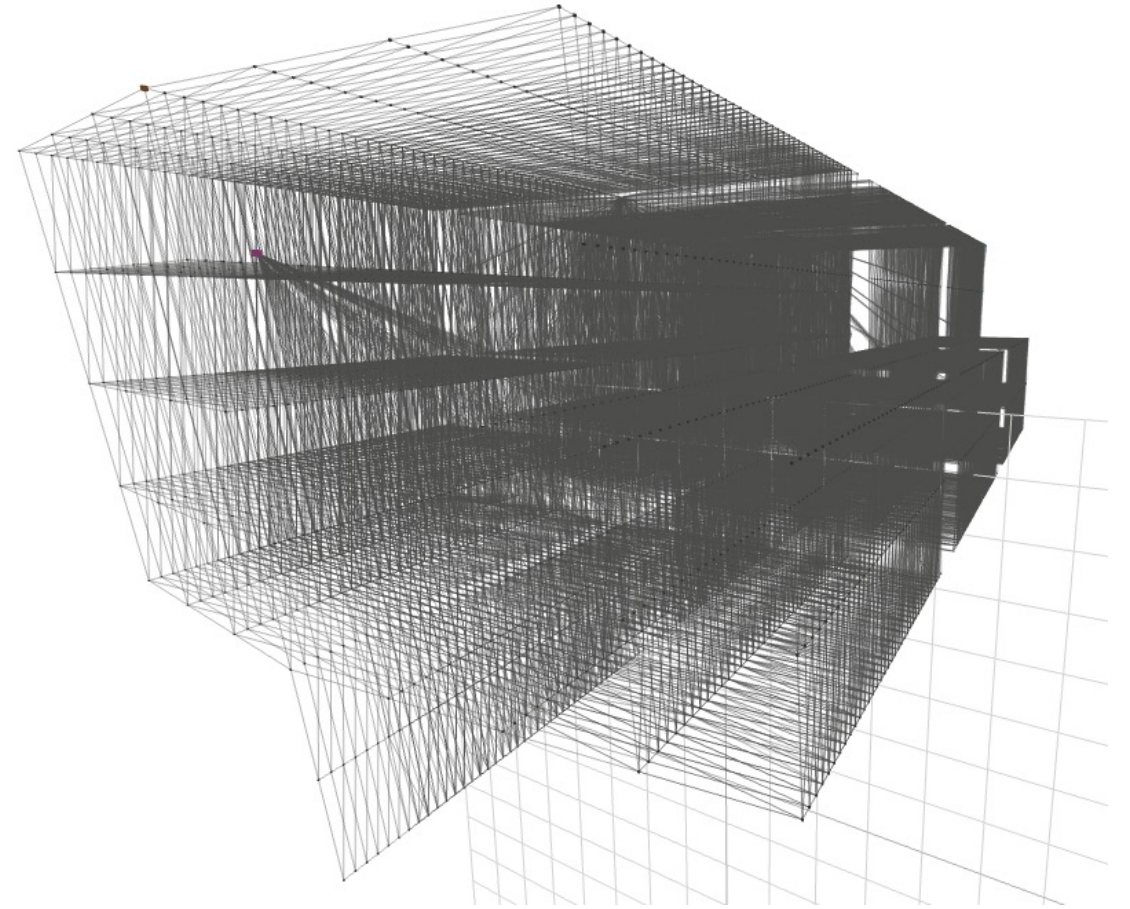
Scoring: Relative Human Action Efficiency (RHAE)

- For each level the agent completes, count its actions a and compare with the human baseline h (upper-median best human action count)
- Level score $S = \min(1.15, (h / a)^2)$ — e.g. a human needs 10 actions and the agent 100 $\rightarrow (10/100)^2 = 1\%$
- Environment score = linearly weighted average of level scores (with a cap); benchmark score = average over environments — **100% = as efficient as humans on every game**
- Semi-private leaderboard at release: Claude Opus 4.6 (Max) 0.50%, Gemini 3.1 Pro Preview 0.40%, GPT-5.4 (High) 0.20%, Grok-4.20 0.10%



Why ARC-AGI-3 is a harness problem

- A raw LLM sees one frame at a time; it has no memory of what it tried, no model of the game, no exploration strategy and no action budget — all of these live in the harness
- Early harness research: graph-based exploration of the state space (Dec 2025), 'explore before you solve' epistemic agents (May 2026), programmatic world modeling (OPINE-World, Jul 2026)
- Later levels compose concepts from earlier ones — the agent must carry what it learned forward (continual learning)
- Course links: exploration and sparse reward (Lecture 11a), world models (Lecture 10b), test-time compute and looping (Lectures 8b, 12), test-time training



Summary and pointers

- An agent = model + harness; the harness supplies the loop, tools, memory, permissions and verification, and its pieces keep migrating into the model
- Inference-time search only pays with a verifier — reward models, tests, process supervision
- ARC-AGI-3 measures **skill-acquisition efficiency** in novel interactive worlds: humans 100%, frontier models < 1%; progress will come from exploration, memory and world models in the harness as much as from the model
- Watch/read: YC Paper Club, *Why the harness matters more than the model* (2026); Raschka & Bowne-Anderson interview (2026); arcprize.org/arc-agi/3 and the technical report (in the sources folder)